E-commercce Checkout Drop-off Data analysis

```sql
-- SQLBook: Code
CREATE INDEX idx_ecommerce_session
ON ecommerce_events(user_session);
-- SQLBook: Code
CREATE INDEX idx_ecommerce_event_type
ON ecommerce_events(event_type);
-- SQLBook: Code
SELECT
    indexname,
    indexdef
FROM pg_indexes
WHERE tablename = 'ecommerce_events';
-- SQLBook: Code
CREATE TABLE session_summary AS
SELECT
    user_session,

    MAX(user_id) AS user_id,

    MIN(event_time) AS session_start,

    MAX(event_time) AS session_end,

    COUNT(*) AS total_events,

    COUNT(*) FILTER (
        WHERE event_type = 'view'
    ) AS product_views,

    COUNT(*) FILTER (
        WHERE event_type = 'cart'
    ) AS cart_adds,

    COUNT(*) FILTER (
        WHERE event_type = 'purchase'
    ) AS purchases

FROM ecommerce_events

GROUP BY user_session;
-- SQLBook: Code
SELECT COUNT(*) AS total_sessions
FROM session_summary;
-- SQLBook: Code
SELECT *
FROM session_summary
LIMIT 10;
-- SQLBook: Code
CREATE INDEX idx_session_summary_user
ON session_summary(user_id);
```

```sql
-- SQLBook: Code
CREATE INDEX idx_session_summary_funnel
ON session_summary(cart_adds, purchases);
-- SQLBook: Code
SELECT
    COUNT(*) AS total_sessions,

    COUNT(*) FILTER (
        WHERE product_views > 0
    ) AS view_sessions,

    COUNT(*) FILTER (
        WHERE cart_adds > 0
    ) AS cart_sessions,

    COUNT(*) FILTER (
        WHERE purchases > 0
    ) AS purchase_sessions

FROM session_summary;
-- SQLBook: Code
SELECT

    COUNT(*) AS total_sessions,

    COUNT(*) FILTER (
        WHERE product_views > 0
    ) AS view_sessions,

    COUNT(*) FILTER (
        WHERE cart_adds > 0
    ) AS cart_sessions,

    COUNT(*) FILTER (
        WHERE purchases > 0
    ) AS purchase_sessions,

    ROUND(
        100.0 *
        COUNT(*) FILTER (
            WHERE cart_adds > 0
        )
        /
        NULLIF(
            COUNT(*) FILTER (
                WHERE product_views > 0
            ),
            0
        ),
        2
    ) AS add_to_cart_rate,

    ROUND(
        100.0 *
```

```
        COUNT(*) FILTER (
            WHERE purchases > 0
        )
        /
        NULLIF(
            COUNT(*) FILTER (
                WHERE cart_adds > 0
            ),
            0
        ),
        2
    ) AS cart_to_purchase_rate,

    ROUND(
        100.0 *
        COUNT(*) FILTER (
            WHERE cart_adds > 0
            AND purchases = 0
        )
        /
        NULLIF(
            COUNT(*) FILTER (
                WHERE cart_adds > 0
            ),
            0
        ),
        2
    ) AS cart_abandonment_rate

FROM session_summary;
-- SQLBook: Code
SELECT
    COUNT(*) FILTER (
        WHERE cart_adds > 0
        AND purchases > 0
    ) AS cart_and_purchase_sessions,

    COUNT(*) FILTER (
        WHERE cart_adds > 0
        AND purchases = 0
    ) AS abandoned_cart_sessions,

    COUNT(*) FILTER (
        WHERE cart_adds = 0
        AND purchases > 0
    ) AS purchase_without_cart_sessions

FROM session_summary;
-- SQLBook: Code
CREATE TABLE session_summary_v2 AS
SELECT
    user_session,

    MAX(user_id) AS user_id,
```

```sql

    MIN(event_time) AS session_start,

    MAX(event_time) AS session_end,

    COUNT(*) AS total_events,

    COUNT(*) FILTER (
        WHERE event_type = 'view'
    ) AS product_views,

    COUNT(*) FILTER (
        WHERE event_type = 'cart'
    ) AS cart_adds,

    COUNT(*) FILTER (
        WHERE event_type = 'purchase'
    ) AS purchases,

    MIN(event_time) FILTER (
        WHERE event_type = 'view'
    ) AS first_view_time,

    MIN(event_time) FILTER (
        WHERE event_type = 'cart'
    ) AS first_cart_time,

    MIN(event_time) FILTER (
        WHERE event_type = 'purchase'
    ) AS first_purchase_time

FROM ecommerce_events

GROUP BY user_session;
-- SQLBook: Code
SELECT *
FROM session_summary_v2
LIMIT 5;
-- SQLBook: Code
SELECT COUNT(*) AS total_sessions
FROM session_summary_v2;
-- SQLBook: Code
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'session_summary_v2'
ORDER BY ordinal_position;
-- SQLBook: Code
ALTER TABLE session_summary_v2
ADD COLUMN funnel_stage VARCHAR(50);
-- SQLBook: Code
UPDATE session_summary_v2
SET funnel_stage =
    CASE
        WHEN first_cart_time IS NOT NULL
```

```
             AND first_purchase_time IS NOT NULL
             AND first_purchase_time > first_cart_time
            THEN 'Cart → Purchase'

        WHEN first_cart_time IS NOT NULL
             AND first_purchase_time IS NULL
            THEN 'Cart Abandoned'

        WHEN first_cart_time IS NULL
             AND first_purchase_time IS NOT NULL
            THEN 'Purchase Without Cart'

        WHEN first_view_time IS NOT NULL
            THEN 'Viewed Only'

        ELSE 'Other'
    END;
-- SQLBook: Code
SELECT
    funnel_stage,
    COUNT(*) AS sessions
FROM session_summary_v2
GROUP BY funnel_stage
ORDER BY sessions DESC;
-- SQLBook: Code
SELECT

    COUNT(*) AS total_sessions,

    COUNT(*) FILTER (
        WHERE funnel_stage = 'Viewed Only'
    ) AS viewed_only_sessions,

    COUNT(*) FILTER (
        WHERE funnel_stage = 'Cart → Purchase'
    ) AS converted_cart_sessions,

    COUNT(*) FILTER (
        WHERE funnel_stage = 'Cart Abandoned'
    ) AS abandoned_cart_sessions,

    COUNT(*) FILTER (
        WHERE funnel_stage = 'Purchase Without Cart'
    ) AS purchase_without_cart_sessions,

    ROUND(
        100.0 *
        COUNT(*) FILTER (
            WHERE funnel_stage = 'Cart → Purchase'
        )
        /
        NULLIF(
            COUNT(*) FILTER (
                WHERE funnel_stage = 'Cart → Purchase'
```

```
                    OR funnel_stage = 'Cart Abandoned'
            ),
            0
        ),
        2
    ) AS cart_to_purchase_rate,

    ROUND(
        100.0 *
        COUNT(*) FILTER (
            WHERE funnel_stage = 'Cart Abandoned'
        )
        /
        NULLIF(
            COUNT(*) FILTER (
                WHERE funnel_stage = 'Cart → Purchase'
                   OR funnel_stage = 'Cart Abandoned'
            ),
            0
        ),
        2
    ) AS cart_abandonment_rate

FROM session_summary_v2;
-- SQLBook: Code
SELECT
    COUNT(*) AS purchase_before_cart_sessions
FROM session_summary_v2
WHERE first_purchase_time IS NOT NULL
  AND first_cart_time IS NOT NULL
  AND first_purchase_time < first_cart_time;
-- SQLBook: Code
SELECT
    funnel_stage,

    ROUND(
        AVG(
            EXTRACT(
                EPOCH FROM (session_end - session_start)
            ) / 60
        ),
        2
    ) AS avg_session_minutes

FROM session_summary_v2

GROUP BY funnel_stage
ORDER BY avg_session_minutes DESC;
-- SQLBook: Code
SELECT
    funnel_stage,

    ROUND(AVG(product_views), 2) AS avg_product_views,

```

```
    ROUND(AVG(cart_adds), 2) AS avg_cart_adds,

    ROUND(AVG(purchases), 2) AS avg_purchases,

    ROUND(AVG(total_events), 2) AS avg_total_events

FROM session_summary_v2

GROUP BY funnel_stage
ORDER BY funnel_stage;
-- SQLBook: Code
SELECT
    e.category_code,
    COUNT(DISTINCT e.user_session) AS cart_sessions
FROM ecommerce_events e
WHERE e.event_type = 'cart'
  AND e.category_code IS NOT NULL
GROUP BY e.category_code
ORDER BY cart_sessions DESC
LIMIT 20;
-- SQLBook: Code
SELECT
    e.category_code,

    COUNT(DISTINCT e.user_session) AS abandoned_sessions

FROM ecommerce_events e

INNER JOIN session_summary_v2 s
    ON e.user_session = s.user_session

WHERE e.event_type = 'cart'
  AND s.funnel_stage = 'Cart Abandoned'
  AND e.category_code IS NOT NULL

GROUP BY e.category_code

ORDER BY abandoned_sessions DESC

LIMIT 20;
-- SQLBook: Code
SELECT
    e.category_code,

    COUNT(DISTINCT e.user_session) AS total_cart_sessions,

    COUNT(DISTINCT CASE
        WHEN s.funnel_stage = 'Cart Abandoned'
        THEN e.user_session
    END) AS abandoned_sessions,

    COUNT(DISTINCT CASE
        WHEN s.funnel_stage = 'Cart → Purchase'
        THEN e.user_session
```

```sql
    END) AS converted_sessions,

    ROUND(
        100.0 *
        COUNT(DISTINCT CASE
            WHEN s.funnel_stage = 'Cart Abandoned'
            THEN e.user_session
        END)
        /
        NULLIF(
            COUNT(DISTINCT e.user_session),
            0
        ),
        2
    ) AS abandonment_rate

FROM ecommerce_events e

INNER JOIN session_summary_v2 s
    ON e.user_session = s.user_session

WHERE e.event_type = 'cart'
  AND e.category_code IS NOT NULL
  AND s.funnel_stage IN (
      'Cart Abandoned',
      'Cart → Purchase'
  )

GROUP BY e.category_code

HAVING COUNT(DISTINCT e.user_session) >= 100

ORDER BY abandonment_rate DESC;
-- SQLBook: Code
SELECT
    CASE
        WHEN e.price < 50 THEN 'Under $50'
        WHEN e.price < 100 THEN '$50-$99'
        WHEN e.price < 250 THEN '$100-$249'
        WHEN e.price < 500 THEN '$250-$499'
        WHEN e.price < 1000 THEN '$500-$999'
        ELSE '$1,000+'
    END AS price_bucket,

    COUNT(DISTINCT e.user_session) AS cart_sessions,

    COUNT(DISTINCT CASE
        WHEN s.funnel_stage = 'Cart Abandoned'
        THEN e.user_session
    END) AS abandoned_sessions,

    COUNT(DISTINCT CASE
        WHEN s.funnel_stage = 'Cart → Purchase'
        THEN e.user_session
```

```sql
    END) AS converted_sessions,

    ROUND(
        100.0 *
        COUNT(DISTINCT CASE
            WHEN s.funnel_stage = 'Cart Abandoned'
            THEN e.user_session
        END)
        /
        NULLIF(
            COUNT(DISTINCT e.user_session),
            0
        ),
        2
    ) AS abandonment_rate

FROM ecommerce_events e

INNER JOIN session_summary_v2 s
    ON e.user_session = s.user_session

WHERE e.event_type = 'cart'
  AND e.price IS NOT NULL
  AND s.funnel_stage IN (
      'Cart Abandoned',
      'Cart → Purchase'
  )

GROUP BY
    CASE
        WHEN e.price < 50 THEN 'Under $50'
        WHEN e.price < 100 THEN '$50-$99'
        WHEN e.price < 250 THEN '$100-$249'
        WHEN e.price < 500 THEN '$250-$499'
        WHEN e.price < 1000 THEN '$500-$999'
        ELSE '$1,000+'
    END

ORDER BY MIN(e.price);
-- SQLBook: Code
SELECT
    CASE
        WHEN e.price < 50 THEN 'Under $50'
        WHEN e.price < 100 THEN '$50-$99'
        WHEN e.price < 250 THEN '$100-$249'
        WHEN e.price < 500 THEN '$250-$499'
        WHEN e.price < 1000 THEN '$500-$999'
        ELSE '$1,000+'
    END AS price_bucket,

    COUNT(DISTINCT e.user_session) AS cart_sessions,

    COUNT(DISTINCT CASE
        WHEN s.funnel_stage = 'Cart Abandoned'
```

```
        THEN e.user_session
    END) AS actual_abandoned,

    ROUND(
        COUNT(DISTINCT e.user_session) * 0.5005
    ) AS expected_abandoned,

    COUNT(DISTINCT CASE
        WHEN s.funnel_stage = 'Cart Abandoned'
        THEN e.user_session
    END)
    -
    ROUND(
        COUNT(DISTINCT e.user_session) * 0.5005
    ) AS excess_abandoned

FROM ecommerce_events e

INNER JOIN session_summary_v2 s
    ON e.user_session = s.user_session

WHERE e.event_type = 'cart'
  AND e.price IS NOT NULL
  AND s.funnel_stage IN (
      'Cart Abandoned',
      'Cart → Purchase'
  )

GROUP BY
    CASE
        WHEN e.price < 50 THEN 'Under $50'
        WHEN e.price < 100 THEN '$50-$99'
        WHEN e.price < 250 THEN '$100-$249'
        WHEN e.price < 500 THEN '$250-$499'
        WHEN e.price < 1000 THEN '$500-$999'
        ELSE '$1,000+'
    END

ORDER BY excess_abandoned DESC;
-- SQLBook: Code
SELECT
    e.category_code,

    ROUND(AVG(e.price), 2) AS avg_cart_price,

    COUNT(DISTINCT e.user_session) AS cart_sessions,

    COUNT(DISTINCT CASE
        WHEN s.funnel_stage = 'Cart Abandoned'
        THEN e.user_session
    END) AS abandoned_sessions,

    ROUND(
        100.0 *
```

```
        COUNT(DISTINCT CASE
            WHEN s.funnel_stage = 'Cart Abandoned'
            THEN e.user_session
        END)
        /
        NULLIF(
            COUNT(DISTINCT e.user_session),
            0
        ),
        2
    ) AS abandonment_rate

FROM ecommerce_events e

INNER JOIN session_summary_v2 s
    ON e.user_session = s.user_session

WHERE e.event_type = 'cart'
  AND e.category_code IS NOT NULL
  AND e.price IS NOT NULL
  AND s.funnel_stage IN (
      'Cart Abandoned',
      'Cart → Purchase'
  )

GROUP BY e.category_code

HAVING COUNT(DISTINCT e.user_session) >= 500

ORDER BY abandonment_rate DESC;
-- SQLBook: Code
SELECT
    CASE
        WHEN cart_adds = 1 THEN '1 item'
        WHEN cart_adds = 2 THEN '2 items'
        WHEN cart_adds = 3 THEN '3 items'
        WHEN cart_adds BETWEEN 4 AND 5 THEN '4-5 items'
        ELSE '6+ items'
    END AS cart_size,

    COUNT(*) AS cart_sessions,

    COUNT(*) FILTER (
        WHERE funnel_stage = 'Cart Abandoned'
    ) AS abandoned_sessions,

    COUNT(*) FILTER (
        WHERE funnel_stage = 'Cart → Purchase'
    ) AS converted_sessions,

    ROUND(
        100.0 *
        COUNT(*) FILTER (
            WHERE funnel_stage = 'Cart Abandoned'
```

```
        )
        /
        NULLIF(COUNT(*), 0),
        2
    ) AS abandonment_rate

FROM session_summary_v2

WHERE funnel_stage IN (
    'Cart Abandoned',
    'Cart → Purchase'
)

GROUP BY
    CASE
        WHEN cart_adds = 1 THEN '1 item'
        WHEN cart_adds = 2 THEN '2 items'
        WHEN cart_adds = 3 THEN '3 items'
        WHEN cart_adds BETWEEN 4 AND 5 THEN '4-5 items'
        ELSE '6+ items'
    END

ORDER BY
    MIN(cart_adds);
-- SQLBook: Code
SELECT
    EXTRACT(HOUR FROM first_cart_time) AS hour,

    COUNT(*) AS cart_sessions,

    COUNT(*) FILTER (
        WHERE funnel_stage = 'Cart Abandoned'
    ) AS abandoned_sessions,

    COUNT(*) FILTER (
        WHERE funnel_stage = 'Cart → Purchase'
    ) AS converted_sessions,

    ROUND(
        100.0 *
        COUNT(*) FILTER (
            WHERE funnel_stage = 'Cart Abandoned'
        )
        /
        NULLIF(COUNT(*), 0),
        2
    ) AS abandonment_rate

FROM session_summary_v2

WHERE funnel_stage IN (
    'Cart Abandoned',
    'Cart → Purchase'
)
```

```sql

GROUP BY EXTRACT(HOUR FROM first_cart_time)

ORDER BY hour;
-- SQLBook: Code
SELECT
    CASE
        WHEN EXTRACT(HOUR FROM first_cart_time) BETWEEN 0 AND 5
            THEN 'Night (00-05)'

        WHEN EXTRACT(HOUR FROM first_cart_time) BETWEEN 6 AND 11
            THEN 'Morning (06-11)'

        WHEN EXTRACT(HOUR FROM first_cart_time) BETWEEN 12 AND 17
            THEN 'Afternoon (12-17)'

        ELSE 'Evening (18-23)'
    END AS time_period,

    COUNT(*) AS cart_sessions,

    COUNT(*) FILTER (
        WHERE funnel_stage = 'Cart Abandoned'
    ) AS abandoned_sessions,

    COUNT(*) FILTER (
        WHERE funnel_stage = 'Cart → Purchase'
    ) AS converted_sessions,

    ROUND(
        100.0 *
        COUNT(*) FILTER (
            WHERE funnel_stage = 'Cart Abandoned'
        )
        /
        NULLIF(COUNT(*), 0),
        2
    ) AS abandonment_rate

FROM session_summary_v2

WHERE funnel_stage IN (
    'Cart Abandoned',
    'Cart → Purchase'
)

GROUP BY
    CASE
        WHEN EXTRACT(HOUR FROM first_cart_time) BETWEEN 0 AND 5
            THEN 'Night (00-05)'

        WHEN EXTRACT(HOUR FROM first_cart_time) BETWEEN 6 AND 11
            THEN 'Morning (06-11)'

```

```
        WHEN EXTRACT(HOUR FROM first_cart_time) BETWEEN 12 AND 17
            THEN 'Afternoon (12-17)'

        ELSE 'Evening (18-23)'
    END

ORDER BY
    MIN(EXTRACT(HOUR FROM first_cart_time));
-- SQLBook: Code
SELECT
    CASE
        WHEN product_views = 1 THEN '1 view'
        WHEN product_views = 2 THEN '2 views'
        WHEN product_views = 3 THEN '3 views'
        WHEN product_views BETWEEN 4 AND 5 THEN '4-5 views'
        WHEN product_views BETWEEN 6 AND 10 THEN '6-10 views'
        ELSE '11+ views'
    END AS product_view_bucket,

    COUNT(*) AS cart_sessions,

    COUNT(*) FILTER (
        WHERE funnel_stage = 'Cart Abandoned'
    ) AS abandoned_sessions,

    COUNT(*) FILTER (
        WHERE funnel_stage = 'Cart → Purchase'
    ) AS converted_sessions,

    ROUND(
        100.0 *
        COUNT(*) FILTER (
            WHERE funnel_stage = 'Cart Abandoned'
        )
        /
        NULLIF(COUNT(*), 0),
        2
    ) AS abandonment_rate

FROM session_summary_v2

WHERE funnel_stage IN (
    'Cart Abandoned',
    'Cart → Purchase'
)

GROUP BY
    CASE
        WHEN product_views = 1 THEN '1 view'
        WHEN product_views = 2 THEN '2 views'
        WHEN product_views = 3 THEN '3 views'
        WHEN product_views BETWEEN 4 AND 5 THEN '4-5 views'
        WHEN product_views BETWEEN 6 AND 10 THEN '6-10 views'
        ELSE '11+ views'
```

```sql
    END

ORDER BY
    MIN(product_views);
-- SQLBook: Code
WITH session_product_behavior AS (

    SELECT
        user_session,

        COUNT(*) AS total_product_views,

        COUNT(DISTINCT product_id) AS unique_products_viewed

    FROM ecommerce_events

    WHERE event_type = 'view'
      AND user_session IS NOT NULL
      AND product_id IS NOT NULL

    GROUP BY user_session
)

SELECT
    CASE
        WHEN total_product_views = unique_products_viewed
            THEN 'No Repetition'

        WHEN total_product_views - unique_products_viewed = 1
            THEN '1 Repeat View'

        WHEN total_product_views - unique_products_viewed BETWEEN 2 AND 3
            THEN '2-3 Repeat Views'

        WHEN total_product_views - unique_products_viewed BETWEEN 4 AND 5
            THEN '4-5 Repeat Views'

        ELSE '6+ Repeat Views'
    END AS repetition_bucket,

    COUNT(*) AS sessions,

    COUNT(*) FILTER (
        WHERE s.funnel_stage = 'Cart Abandoned'
    ) AS abandoned_sessions,

    COUNT(*) FILTER (
        WHERE s.funnel_stage = 'Cart → Purchase'
    ) AS converted_sessions,

    ROUND(
        100.0 *
        COUNT(*) FILTER (
            WHERE s.funnel_stage = 'Cart Abandoned'
```

```sql
        )
        /
        NULLIF(COUNT(*), 0),
        2
    ) AS abandonment_rate

FROM session_product_behavior p

INNER JOIN session_summary_v2 s
    ON p.user_session = s.user_session

WHERE s.funnel_stage IN (
    'Cart Abandoned',
    'Cart → Purchase'
)

GROUP BY
    CASE
        WHEN total_product_views = unique_products_viewed
            THEN 'No Repetition'

        WHEN total_product_views - unique_products_viewed = 1
            THEN '1 Repeat View'

        WHEN total_product_views - unique_products_viewed BETWEEN 2 AND 3
            THEN '2-3 Repeat Views'

        WHEN total_product_views - unique_products_viewed BETWEEN 4 AND 5
            THEN '4-5 Repeat Views'

        ELSE '6+ Repeat Views'
    END

ORDER BY
    MIN(total_product_views - unique_products_viewed);
-- SQLBook: Code
WITH session_product_behavior AS (

    SELECT
        user_session,

        COUNT(*) AS total_product_views,

        COUNT(DISTINCT product_id) AS unique_products_viewed,

        COUNT(*) - COUNT(DISTINCT product_id) AS repeat_views

    FROM ecommerce_events

    WHERE event_type = 'view'
      AND user_session IS NOT NULL
      AND product_id IS NOT NULL

    GROUP BY user_session
```

```sql
),

category_session_behavior AS (

    SELECT DISTINCT
        e.user_session,
        e.category_code
    FROM ecommerce_events e
    WHERE e.event_type = 'cart'
      AND e.category_code IS NOT NULL
)

SELECT
    c.category_code,

    COUNT(DISTINCT c.user_session) AS cart_sessions,

    ROUND(
        AVG(p.repeat_views),
        2
    ) AS avg_repeat_views,

    COUNT(DISTINCT CASE
        WHEN p.repeat_views >= 4
        THEN c.user_session
    END) AS high_repetition_sessions,

    ROUND(
        100.0 *
        COUNT(DISTINCT CASE
            WHEN p.repeat_views >= 4
            THEN c.user_session
        END)
        /
        NULLIF(
            COUNT(DISTINCT c.user_session),
            0
        ),
        2
    ) AS high_repetition_rate,

    COUNT(DISTINCT CASE
        WHEN s.funnel_stage = 'Cart Abandoned'
        THEN c.user_session
    END) AS abandoned_sessions,

    ROUND(
        100.0 *
        COUNT(DISTINCT CASE
            WHEN s.funnel_stage = 'Cart Abandoned'
            THEN c.user_session
        END)
        /
        NULLIF(
```

```
            COUNT(DISTINCT c.user_session),
            0
        ),
        2
    ) AS abandonment_rate

FROM category_session_behavior c

INNER JOIN session_product_behavior p
    ON c.user_session = p.user_session

INNER JOIN session_summary_v2 s
    ON c.user_session = s.user_session

WHERE s.funnel_stage IN (
    'Cart Abandoned',
    'Cart → Purchase'
)

GROUP BY c.category_code

HAVING COUNT(DISTINCT c.user_session) >= 500

ORDER BY high_repetition_rate DESC;
-- SQLBook: Code
WITH session_product_behavior AS (
    SELECT
        user_session,
        COUNT(*) AS total_product_views,
        COUNT(DISTINCT product_id) AS unique_products_viewed,
        COUNT(*) - COUNT(DISTINCT product_id) AS repeat_views
    FROM ecommerce_events
    WHERE event_type = 'view'
      AND user_session IS NOT NULL
      AND product_id IS NOT NULL
    GROUP BY user_session
),

category_session_behavior AS (
    SELECT DISTINCT
        user_session,
        category_code
    FROM ecommerce_events
    WHERE event_type = 'cart'
      AND category_code IS NOT NULL
),

category_metrics AS (
    SELECT
        c.category_code,

        COUNT(DISTINCT c.user_session) AS cart_sessions,

        100.0 *
```

```
        COUNT(DISTINCT CASE
            WHEN s.funnel_stage = 'Cart Abandoned'
            THEN c.user_session
        END)
        /
        NULLIF(COUNT(DISTINCT c.user_session), 0)
        AS abandonment_rate,

        100.0 *
        COUNT(DISTINCT CASE
            WHEN p.repeat_views >= 4
            THEN c.user_session
        END)
        /
        NULLIF(COUNT(DISTINCT c.user_session), 0)
        AS high_repetition_rate

    FROM category_session_behavior c

    JOIN session_product_behavior p
        ON c.user_session = p.user_session

    JOIN session_summary_v2 s
        ON c.user_session = s.user_session

    WHERE s.funnel_stage IN (
        'Cart Abandoned',
        'Cart → Purchase'
    )

    GROUP BY c.category_code

    HAVING COUNT(DISTINCT c.user_session) >= 500
),

min_max AS (
    SELECT
        MIN(abandonment_rate) AS min_abandonment,
        MAX(abandonment_rate) AS max_abandonment,

        MIN(high_repetition_rate) AS min_repetition,
        MAX(high_repetition_rate) AS max_repetition,

        MIN(cart_sessions) AS min_volume,
        MAX(cart_sessions) AS max_volume

    FROM category_metrics
),

scored_categories AS (
    SELECT
        c.category_code,
        c.cart_sessions,
        c.abandonment_rate,
```

```
        c.high_repetition_rate,

        100.0 *
        (c.abandonment_rate - m.min_abandonment)
        /
        NULLIF(
            m.max_abandonment - m.min_abandonment,
            0
        ) AS abandonment_score,

        100.0 *
        (c.high_repetition_rate - m.min_repetition)
        /
        NULLIF(
            m.max_repetition - m.min_repetition,
            0
        ) AS repetition_score,

        100.0 *
        (c.cart_sessions - m.min_volume)
        /
        NULLIF(
            m.max_volume - m.min_volume,
            0
        ) AS volume_score

    FROM category_metrics c
    CROSS JOIN min_max m
)

SELECT
    category_code,
    cart_sessions,

    ROUND(abandonment_rate::numeric, 2)
        AS abandonment_rate,

    ROUND(high_repetition_rate::numeric, 2)
        AS high_repetition_rate,

    ROUND(abandonment_score::numeric, 2)
        AS abandonment_score,

    ROUND(repetition_score::numeric, 2)
        AS repetition_score,

    ROUND(volume_score::numeric, 2)
        AS volume_score,

    ROUND(
        (
            abandonment_score * 0.40
            + repetition_score * 0.30
            + volume_score * 0.30
```

```
        )::numeric,
        2
    ) AS priority_score

FROM scored_categories

ORDER BY priority_score DESC;
-- SQLBook: Code
DROP TABLE IF EXISTS category_metrics;

CREATE TABLE category_metrics AS
WITH session_product_behavior AS (
    SELECT
        user_session,
        COUNT(*) AS total_product_views,
        COUNT(DISTINCT product_id) AS unique_products_viewed,
        COUNT(*) - COUNT(DISTINCT product_id) AS repeat_views
    FROM ecommerce_events
    WHERE event_type = 'view'
      AND user_session IS NOT NULL
      AND product_id IS NOT NULL
    GROUP BY user_session
),

category_session_behavior AS (
    SELECT DISTINCT
        user_session,
        category_code
    FROM ecommerce_events
    WHERE event_type = 'cart'
      AND category_code IS NOT NULL
)
SELECT
    c.category_code,

    COUNT(DISTINCT c.user_session) AS cart_sessions,

    ROUND(
        (
            100.0 *
            COUNT(DISTINCT CASE
                WHEN s.funnel_stage = 'Cart Abandoned'
                THEN c.user_session
            END)
            /
            NULLIF(COUNT(DISTINCT c.user_session), 0)
        )::numeric,
        2
    ) AS abandonment_rate,

    ROUND(
        (
            100.0 *
            COUNT(DISTINCT CASE
```

```
                WHEN p.repeat_views >= 4
                THEN c.user_session
            END)
            /
            NULLIF(COUNT(DISTINCT c.user_session), 0)
        )::numeric,
        2
    ) AS high_repetition_rate

FROM category_session_behavior c

JOIN session_product_behavior p
    ON c.user_session = p.user_session

JOIN session_summary_v2 s
    ON c.user_session = s.user_session

WHERE s.funnel_stage IN (
    'Cart Abandoned',
    'Cart → Purchase'
)

GROUP BY c.category_code

HAVING COUNT(DISTINCT c.user_session) >= 500;
-- SQLBook: Code
SELECT *
FROM category_metrics
ORDER BY abandonment_rate DESC;
-- SQLBook: Code
WITH metrics AS (

    SELECT
        category_code,
        cart_sessions,
        abandonment_rate,
        high_repetition_rate,

        ROUND(
            cart_sessions * abandonment_rate / 100.0
        ) AS abandoned_sessions

    FROM category_metrics
),

scored AS (

    SELECT
        *,

        100.0 *
        (
            abandoned_sessions
            - MIN(abandoned_sessions) OVER ()
```

```
        )
        /
        NULLIF(
            MAX(abandoned_sessions) OVER ()
            - MIN(abandoned_sessions) OVER (),
            0
        ) AS abandoned_volume_score,

        100.0 *
        (
            abandonment_rate
            - MIN(abandonment_rate) OVER ()
        )
        /
        NULLIF(
            MAX(abandonment_rate) OVER ()
            - MIN(abandonment_rate) OVER (),
            0
        ) AS abandonment_score,

        100.0 *
        (
            high_repetition_rate
            - MIN(high_repetition_rate) OVER ()
        )
        /
        NULLIF(
            MAX(high_repetition_rate) OVER ()
            - MIN(high_repetition_rate) OVER (),
            0
        ) AS repetition_score

    FROM metrics
)

SELECT

    category_code,

    cart_sessions,

    abandoned_sessions,

    abandonment_rate,

    high_repetition_rate,

    ROUND(abandoned_volume_score::numeric, 2)
        AS abandoned_volume_score,

    ROUND(
        (
            abandoned_volume_score * 0.50
            +
```

```
            abandonment_score * 0.30
            +
            repetition_score * 0.20
        )::numeric,
        2
    ) AS opportunity_score

FROM scored

ORDER BY opportunity_score DESC;
-- SQLBook: Code
SELECT
    category_code,
    cart_sessions,
    abandoned_sessions,
    abandonment_rate,
    high_repetition_rate,
    abandoned_volume_score,
    opportunity_score
FROM (
    WITH metrics AS (
        SELECT
            category_code,
            cart_sessions,
            abandonment_rate,
            high_repetition_rate,
            ROUND(
                cart_sessions * abandonment_rate / 100.0
            ) AS abandoned_sessions
        FROM category_metrics
    ),

    scored AS (
        SELECT
            *,
            100.0 *
            (
                abandoned_sessions
                - MIN(abandoned_sessions) OVER ()
            )
            /
            NULLIF(
                MAX(abandoned_sessions) OVER ()
                - MIN(abandoned_sessions) OVER (),
                0
            ) AS abandoned_volume_score,

            100.0 *
            (
                abandonment_rate
                - MIN(abandonment_rate) OVER ()
            )
            /
            NULLIF(
```

```
                MAX(abandonment_rate) OVER ()
                - MIN(abandonment_rate) OVER (),
                0
            ) AS abandonment_score,

            100.0 *
            (
                high_repetition_rate
                - MIN(high_repetition_rate) OVER ()
            )
            /
            NULLIF(
                MAX(high_repetition_rate) OVER ()
                - MIN(high_repetition_rate) OVER (),
                0
            ) AS repetition_score

        FROM metrics
    )
    SELECT
        *,
        ROUND(
            (
                abandoned_volume_score * 0.50
                + abandonment_score * 0.30
                + repetition_score * 0.20
            )::numeric,
            2
        ) AS opportunity_score

    FROM scored
) x
ORDER BY opportunity_score DESC
LIMIT 10;
-- SQLBook: Code
SELECT
    category_code,
    cart_sessions,
    abandoned_sessions,
    abandonment_rate,
    high_repetition_rate,

    CASE
        WHEN abandonment_rate >= 55
             AND cart_sessions >= 2000
             AND high_repetition_rate >= 35
            THEN 'Critical Opportunity'

        WHEN abandonment_rate >= 55
             AND high_repetition_rate >= 35
            THEN 'High Friction'

        WHEN cart_sessions >= 2000
             AND abandonment_rate >= 55
```

```
            THEN 'High Volume + High Abandonment'

        WHEN cart_sessions >= 2000
             AND abandonment_rate < 55
            THEN 'High Volume - Monitor'

        ELSE 'Lower Priority'
    END AS business_priority

FROM (
    SELECT
        category_code,
        cart_sessions,

        ROUND(
            cart_sessions * abandonment_rate / 100.0
        ) AS abandoned_sessions,

        abandonment_rate,
        high_repetition_rate

    FROM category_metrics
) x

ORDER BY
    CASE
        WHEN abandonment_rate >= 55
             AND cart_sessions >= 2000
             AND high_repetition_rate >= 35
            THEN 1

        WHEN abandonment_rate >= 55
             AND high_repetition_rate >= 35
            THEN 2

        WHEN cart_sessions >= 2000
             AND abandonment_rate >= 55
            THEN 3

        WHEN cart_sessions >= 2000
             AND abandonment_rate < 55
            THEN 4

        ELSE 5
    END,

    abandonment_rate DESC;
-- SQLBook: Code
DROP TABLE IF EXISTS session_dashboard;

CREATE TABLE session_dashboard AS
SELECT
    s.user_session,

```

```
    s.funnel_stage,

    s.session_start,
    s.session_end,

    EXTRACT(
        EPOCH FROM (s.session_end - s.session_start)
    ) / 60.0 AS session_minutes,

    s.product_views,

    s.cart_adds,

    s.purchases,

    s.total_events,

    CASE
        WHEN s.funnel_stage = 'Cart Abandoned'
        THEN 1
        ELSE 0
    END AS is_abandoned,

    CASE
        WHEN s.funnel_stage = 'Cart → Purchase'
        THEN 1
        ELSE 0
    END AS is_converted

FROM session_summary_v2 s;
-- SQLBook: Code
SELECT
    COUNT(*) AS total_sessions,

    SUM(is_abandoned) AS abandoned_sessions,

    SUM(is_converted) AS converted_sessions,

    ROUND(
        100.0 * SUM(is_abandoned)
        / NULLIF(
            SUM(is_abandoned) + SUM(is_converted),
            0
        ),
        2
    ) AS abandonment_rate

FROM session_dashboard;
-- SQLBook: Code
-- Active: 1787642726036@@127.0.0.1@5432@postgres
CREATE INDEX idx_session_dashboard_stage
ON session_dashboard(funnel_stage);
CREATE INDEX idx_session_dashboard_session
ON session_dashboard(user_session);
```

```sql
-- SQLBook: Code
SELECT
    COUNT(*) AS total_sessions,

    SUM(is_abandoned) AS abandoned_sessions,

    SUM(is_converted) AS converted_sessions,

    ROUND(
        100.0 * SUM(is_abandoned)
        / NULLIF(
            SUM(is_abandoned) + SUM(is_converted),
            0
        ),
        2
    ) AS abandonment_rate

FROM session_dashboard;
-- SQLBook: Code
ALTER TABLE session_dashboard
ADD COLUMN repeat_views INTEGER;
-- SQLBook: Code

```